

SOPER 2023/2024

Práctica 1

Antonio Morono,
Manuel Fernández López,

1. Preguntas sobre la práctica

Como en esta práctica no se especifica ninguna pregunta para responder, pasamos directamente al siguiente apartado.

2. Descripción de los archivos implementados

Aquí explicamos los ficheros adicionales que hemos creado para la realización de la práctica:

miner.c: Este es el archivo principal, el cual contiene todas las funciones necesarias que hemos implementado para resolver el problema.

Makefile: Makefile utilizado para probar el correcto funcionamiento del ejercicio.

3. Descripción de los requisitos

Esta práctica consiste en comenzar a familiarizarnos con el uso de hilos y procesos. Para ello, tenemos que realizar un programa en el cual haya una serie de mineros cuya misión sea resolver un mismo problema en una serie de rondas. Cuando un minero lo encuentra, pasamos a la siguiente ronda.

Para ello, utilizamos hilos, que serán nuestros mineros, varios procesos, los cuales son el principal, el proceso minero y el proceso monitor, siendo el trabajo de cada uno de los procesos distinto de los demás. Además, se comunican entre sí a través de tuberías, familiarizándonos de esta forma con lo más básico del manejo de procesos.

4. Descripción de la implementación

Este programa en C implementa un proceso minero que utiliza múltiples hilos para encontrar la solución a un problema. El problema se define como encontrar un número que, cuando se pasa a través de una función hash (`pow_hash``), es igual a un objetivo dado. El espacio de búsqueda se divide entre los hilos, y cada hilo busca la solución en su intervalo asignado.

La función ``main`` comprueba si se ha pasado el número correcto de argumentos, crea un proceso minero utilizando ``fork`` y espera a que termine.

La función ``miner`` es la función principal del proceso minero. Crea una tubería para la comunicación entre procesos, se bifurca para crear un proceso monitor, y configura los hilos para el proceso minero. A continuación, entra en un bucle donde crea los hilos, espera a que terminen, y se comunica con el proceso monitor a través de la tubería. Si el proceso monitor valida la solución, el objetivo se actualiza y el bucle continúa durante el número especificado de rondas.

La función ``threadf`` es la función que ejecuta cada hilo. Toma como argumento una estructura ``Miner_data``, que contiene el inicio y el final del intervalo que el hilo debe buscar. El hilo busca en su intervalo un número que, una vez obtenido el hash, sea igual al objetivo. Si lo encuentra, actualiza la variable global ``flag`` y sale.

La función `monitor` es la función principal del proceso monitor. Lee las soluciones de la tubería, las valida haciendo un hash de la solución y comparándola con el objetivo, y escribe el resultado de la validación de nuevo a la tubería. Si se encuentra con un error durante la lectura de la tubería, se sale.

5. Pruebas de funcionamiento

Para probar el correcto funcionamiento de nuestro código hemos probado a cambiar los parámetros de entrada para probar distintos casos (una única ronda, un único hilo, varias rondas, varios hilos...) para probar los casos más comunes.

Después hemos probado parámetros para los casos extremos y así ver como reacciona el programa (cero rondas, cero hilos, sin argumentos...). En estos casos el programa se comportaba bien sin bloquearse ni dejar fugas de memoria ni nada por el estilo.

Por último hemos forzado a que el monitor invalide una solución y en este caso el programa se comportó de forma esperada e ideal.

He aquí unos ejemplos de ejecución con la herramienta helgrind:

Ejemplo 1: Caso normal

Running miner with helgrind

==17258== Helgrind, a thread error detector

==17258== Copyright (C) 2007-2017, and GNU GPL'd, by OpenWorks LLP et al.

==17258== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info

==17258== Command: ./mrush 10 10 5

==17258==

Objective: 10

Rounds: 10

Threads: 5

Solución accepted: 00000010 -> 08306237

Solución accepted: 08306237 -> 04869173

Solución accepted: 04869173 -> 04018068

Solución accepted: 04018068 -> 00114153

Solución accepted: 00114153 -> 06394877

Solución accepted: 06394877 -> 04717281

Solución accepted: 04717281 -> 02335279

Solución accepted: 02335279 -> 08738683

Solución accepted: 08738683 -> 08003988

Solución accepted: 08003988 -> 07797366

==17260==

==17260== Use --history-level=approx or =none to gain increased speed, at

==17260== the cost of reduced accuracy of conflicting-access information

==17260== For lists of detected and suppressed errors, rerun with: -s

==17260== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Monitor exited with status 67

==17259==

```

==17259== Use --history-level=approx or =none to gain increased speed, at
==17259== the cost of reduced accuracy of conflicting-access information
==17259== For lists of detected and suppressed errors, rerun with: -s
==17259== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 74 from 22)
Miner exited with status 67
==17258==
==17258== Use --history-level=approx or =none to gain increased speed, at
==17258== the cost of reduced accuracy of conflicting-access information
==17258== For lists of detected and suppressed errors, rerun with: -s
==17258== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Ejemplo 2: Caso extremo

Running miner with helgrind

```

==17420== Helgrind, a thread error detector
==17420== Copyright (C) 2007-2017, and GNU GPL'd, by OpenWorks LLP et al.
==17420== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==17420== Command: ./mrush 0 0 0
==17420==
Objective: 0
Rounds: 0
Threads: 0
==17421==
==17421== Process terminating with default action of signal 8 (SIGFPE)
==17421== Integer divide by zero at address 0x1003205F9F
==17421==    at 0x109707: miner (in /home/ar70n/Documents/SOPER/Practica 1/Ejercicio
13/mrush)
==17421==    by 0x109476: main (in /home/ar70n/Documents/SOPER/Practica 1/Ejercicio
13/mrush)
==17421==
==17421== Use --history-level=approx or =none to gain increased speed, at
==17421== the cost of reduced accuracy of conflicting-access information
==17421== For lists of detected and suppressed errors, rerun with: -s
==17421== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
Miner exited with status 68
==17422==
==17422== Use --history-level=approx or =none to gain increased speed, at
==17422== the cost of reduced accuracy of conflicting-access information
==17422== For lists of detected and suppressed errors, rerun with: -s
==17422== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
==17420==
==17420== Use --history-level=approx or =none to gain increased speed, at
==17420== the cost of reduced accuracy of conflicting-access information
==17420== For lists of detected and suppressed errors, rerun with: -s
==17420== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Ejemplo 3: Caso forzado de error

Running miner with helgrind

==17629== Helgrind, a thread error detector

==17629== Copyright (C) 2007-2017, and GNU GPL'd, by OpenWorks LLP et al.

==17629== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info

==17629== Command: ./mrush 0 2 3

==17629==

Objective: 0

Rounds: 2

Threads: 3

Solución rejected: 00000000 !-> 08916551

The solution has been invalidated

==17630==

==17630== Use --history-level=approx or =none to gain increased speed, at

==17630== the cost of reduced accuracy of conflicting-access information

==17630== For lists of detected and suppressed errors, rerun with: -s

==17630== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 119 from 72)

Miner exited with status 68

==17631==

==17631== Use --history-level=approx or =none to gain increased speed, at

==17631== the cost of reduced accuracy of conflicting-access information

==17631== For lists of detected and suppressed errors, rerun with: -s

==17631== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

==17629==

==17629== Use --history-level=approx or =none to gain increased speed, at

==17629== the cost of reduced accuracy of conflicting-access information

==17629== For lists of detected and suppressed errors, rerun with: -s

==17629== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)